

Lab program-36:

36. With linked allocation, each file is a linked list of disk blocks; the disk blocks may be scattered anywhere on the disk. The directory contains a pointer to the first and last blocks of the file. Each block contains a pointer to the next block. Design a C program to simulate the file allocation strategy.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Block {
    char data[256];
    struct Block *next;
};

int main(void) {
    struct Block *firstBlock = NULL;
    struct Block *lastBlock = NULL;
    int blockCount = 0;
    int blockNumber;
    char data[256];
    char choice;

    printf("Linked Allocation Simulation\n");

    while (1) {
        printf("Enter 'W' to write a block, 'R' to read a block, or 'Q' to quit: ");
        scanf(" %c", &choice);

        if (choice == 'Q' || choice == 'q')
            break;

        if (choice == 'W' || choice == 'w') {
            printf("Enter data for the block: ");
            scanf(" %255[^\n]", data);

            struct Block *newBlock = malloc(sizeof(struct Block));
            if (newBlock == NULL) {
                printf("Memory allocation failed.\n");
                break;
            }
            strcpy(newBlock->data, data);
            newBlock->next = NULL;
```

```

        if (blockCount == 0) {
            firstBlock = newBlock;
            lastBlock = newBlock;
        } else {
            lastBlock->next = newBlock;
            lastBlock = newBlock;
        }
        blockCount++;
    } else if (choice == 'R' || choice == 'r') {
        printf("Enter the block number to read (1-%d): ", blockCount);
        scanf("%d", &blockNumber);

        if (blockNumber < 1 || blockNumber > blockCount) {
            printf("Invalid block number. The valid range is 1-%d.\n", blockCount);
        } else {
            struct Block *currentBlock = firstBlock;
            for (int i = 1; i < blockNumber; i++)
                currentBlock = currentBlock->next;
            printf("Block %d Data: %s\n", blockNumber, currentBlock->data);
        }
    }
}

struct Block *currentBlock = firstBlock;
while (currentBlock != NULL) {
    struct Block *nextBlock = currentBlock->next;
    free(currentBlock);
    currentBlock = nextBlock;
}

return 0;
}

```

OUTPUT:

Linked Allocation Simulation

Enter 'W' to write a block, 'R' to read a block, or 'Q' to quit: W
Enter data for the block: SAI IS WORST
Enter 'W' to write a block, 'R' to read a block, or 'Q' to quit: W
Enter data for the block: JASWANTH IS GOOD
Enter 'W' to write a block, 'R' to read a block, or 'Q' to quit: R
Enter the block number to read (1-2): 2
Block 2 Data: JASWANTH IS GOOD
Enter 'W' to write a block, 'R' to read a block, or 'Q' to quit: R
Enter the block number to read (1-2): 1
Block 1 Data: SAI IS WORST
Enter 'W' to write a block, 'R' to read a block, or 'Q' to quit: Q

Process exited after 46.7 seconds with return value 0
Press any key to continue . . . |